

Одномерные массивы в C++

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
H	e	l	l	o		w	o	r	l	d	\0						

- **Структура** – некоторый составной тип данных, составленный из базовых скалярных.
- Если структура не изменяет своего строения на протяжении всего выполнения программы, в которой она описана, то такую структуру называют **статической**.
- Самой распространенной структурой, реализованной практически во всех языках программирования, является **массив**.

Понятие массива

- В общем случае **массив** – это структурированный тип данных, состоящий из фиксированного числа элементов, имеющих один и тот же тип.
- Название **регулярный тип** массивы получили за то, что в них объединены однотипные (логически однородные) элементы, упорядоченные по индексам, определяющим положение каждого элемента в массиве.

Примеры

- A(1, 5, 9, 3, 17, 12)
- Winter(“December”, “January”, “February”)
- В программировании **массив** – это последовательность однотипных элементов, имеющих общее имя, причем каждый элемент этой последовательности определяется порядковым номером (**индексом**) элемента.
- Индекс – это значение порядкового типа, определенного, как **тип индекса** данного массива.

- Массивы бывают одномерными (один индекс), двумерными (два индекса) и т.д.
- Структура массива всегда однородна. Массив может состоять из элементов типа *int*, *double* или *char*, либо других однотипных элементов.
- К любой компоненте массива можно обращаться произвольным образом. Программа может сразу получить нужный ей элемент по его порядковому номеру (индексу).



Размерность массива

- Размерность массива вместе с типом его элементов определяет объем памяти, необходимый для размещения массива, которое выполняется на этапе компиляции, поэтому размерность может быть задана только **целой положительной константой или константным выражением**.
- Размерность массивов предпочтительнее задавать с помощью именованных констант.

Инициализация массива

- Как и переменные, элементы массива могут быть **инициализированы**.
- Термином "**инициализация**" обозначают возможность задать начальные значения элементов массива без программирования соответствующих действий.
- Например, не прибегая к программным средствам типа присваивания значений в цикле или считывания данных из внешнего источника.

Инициализация массива

- Инициализирующие значения для массивов записываются в фигурных скобках.
- Значения элементам **присваиваются по порядку.**
- Если элементов в массиве больше, чем инициализаторов, элементы, для которых значения не указаны, **обнуляются.**

Описание одномерных массивов в C++

<тип элементов> Имя [размерность];

Примеры:

```
float A[10];
```

```
int b[5] = {3, 2, 1}; // инициализация
```

```
int a[5] = {0}; //обнуление всего массива
```

Нумерация элементов массива в C++
начинается с нуля!

Примеры

- **float a [10];** //размерность задана целой положительной константой
- **const int n = 10;** // размерность задана с помощью именованной константы
- **int marks[n];**
- **int b[] = {3, 2, 1};** // размерность массива равна трем

- Количество элементов массивов при таком описании задаётся непосредственно в исходном коде программы.
- Изменить количество элементов в ходе работы программы таким массивам невозможно. Такие массивы называются ***статически-создаваемыми массивами***.

Алгоритм работы с массивом

- 1) объявление массива;
 - 2) генерация или *инициализация*;
 - 3) обработка значений элементов;
 - 4) *вывод*.
- При этом, если в процессе обработки значения элементов массива изменяются, то **вывод осуществляется дважды**: до и после обработки.

Способы заполнения одномерного массива:

- заполнение одномерного массива по формуле;
- заполнение случайными числами;
- считывание значений элементов массива с клавиатуры;
- считывание элементов одномерного массива из текстового файла.

Заполнение случайными числами

- Функция `srand()`. Синтаксис:

```
void srand(unsigned seed);
```
- функция устанавливает свой аргумент как основу (*seed*) для новой последовательности псевдослучайных целых чисел, возвращаемых функцией *rand()*.
- Сформированную последовательность можно воспроизвести. Для этого необходимо вызвать `srand()` с соответствующей величиной *seed*.
- Для использования данной функции необходимо подключить библиотечный файл **<stdlib.h>**.

- Функция *rand()*. Синтаксис:
int rand(void);
- функция возвращает *псевдослучайное число* в диапазоне от нуля до RAND_MAX.
- **Для использования данной функции необходимо подключить библиотечный файл <stdlib.h>.**
- Константа RAND_MAX определяет максимальное значение случайного числа, которое может быть возвращено функцией *rand()*. Значение RAND_MAX – это максимальное положительное целое число.


```
int i;  
srand(time(NULL) * 1000) ;  
//устанавливает начальную точку  
генерации случайных чисел  
for (i=0; i < k; i++) {  
x[i]=(rand() * 1.0 / (RAND_MAX) * (b-a) + a) ;  
//функция генерации случайных  
// чисел на [a,b)  
}
```